Topic

Enumerations (enum) in C

Programs on explicit enumeration values and menu-driven operations using enum and switch

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

Contents

Sr. No.	Title	Section
1	Introduction to Enumeration	Theory
2	Problem 1: Explicit values starting from 10	Program 1
3	Problem 2: Menu choices using enum and switch	Program 2
4	Conclusion	Summary

1. Introduction to Enumeration

An enumeration is a user-defined data type in C that consists of a set of named integer constants. It is created using the enum keyword. Enumerations make a program easier to read and maintain because meaningful names are used in place of plain numbers.

Syntax

```
enum name {
    constant1,
    constant2,
    constant3
};
```

Important Points

- By default, the first constant in an enum has the value 0, and each following constant increases by 1.
- A programmer can assign an explicit integer value to any constant. The constants that follow continue counting from that value.
- Enum constants are of type int and can be used in expressions, loops and in switch statements.
- Enum constant names must be unique within the same scope.

2. Problem 1: Explicit Values Starting from 10

Problem Statement

Assign explicit values starting from 10 in an enumeration and print them.

Input	Expected Output
Nil	`A=10, B=11, C=12`

Approach / Algorithm

1. Start the program and include the header file stdio.h.
2. Declare an enumeration `Letters` with the constants A, B and C.
3. Assign the explicit value 10 to `A`. B and C automatically become 11 and 12.
4. Print the values of A, B and C using `printf()` with the %d format specifier.
5. Stop the program.

Program

```
#include <stdio.h>

/* Enumeration with explicit values starting from 10 */
enum Letters {
    A = 10,
    B = 11,
    C = 12
};

int main(void)
{
    printf("A=%d, B=%d, C=%d\n", A, B, C);
    return 0;
}
```

Output

```
A=10, B=11, C=12
```

Explanation

In the program, the constant `A` is explicitly set to 10. The values of B and C are also written as 11 and 12, which matches what the compiler would assign automatically after A. Since enum constants are integers, they are printed with the %d format specifier.

3. Problem 2: Menu Choices Using Enum and Switch

Problem Statement

Use an enumeration to represent the menu choices (ADD, SUBTRACT, MULTIPLY) and perform the operations using a switch statement.

Input	Expected Output
`ADD 10 20`	`30`

Approach / Algorithm

1. Start the program and include the header files stdio.h and string.h.
2. Declare an enumeration `Menu` with the constants ADD, SUBTRACT, MULTIPLY and INVALID.
3. Read the operation name and the two numbers from the user.
4. Convert the operation name to its enum value using the function `getChoice()`, which compares strings with strcmp().
5. Pass the enum value to a switch statement. Each case performs the matching operation (+, -, *) and prints the result.
6. If the operation name is not recognised, the default case prints an error message.
7. Stop the program.

Logic Used

C cannot use strings directly as switch case labels, so the text entered by the user is first converted into an enum value. The switch statement then selects the correct operation using the enum constants as case labels. The break statement after every case prevents execution from falling through into the next case, and the default case handles any invalid input.

Program

```
#include <stdio.h>
#include <string.h>

/* Enumeration representing the menu choices */
enum Menu {
    ADD,
    SUBTRACT,
    MULTIPLY,
    INVALID
};

/* Converts the text entered by the user into an enum value */
enum Menu getChoice(const char *text)
{
    if (strcmp(text, "ADD") == 0)
        return ADD;
    else if (strcmp(text, "SUBTRACT") == 0)
        return SUBTRACT;
    else if (strcmp(text, "MULTIPLY") == 0)
        return MULTIPLY;
    else
        return INVALID;
}

int main(void)
{
    char operation[20];
    int num1, num2, result;
    enum Menu choice;

    /* Input format: OPERATION num1 num2  (e.g. ADD 10 20) */
    if (scanf("%19s %d %d", operation, &num1, &num2) != 3) {
        printf("Invalid input\n");
        return 1;
    }

    choice = getChoice(operation);

    switch (choice) {
        case ADD:
            result = num1 + num2;
            printf("%d\n", result);
            break;
        case SUBTRACT:
            result = num1 - num2;
            printf("%d\n", result);
            break;
        case MULTIPLY:
            result = num1 * num2;
            printf("%d\n", result);
            break;
        default:
            printf("Invalid operation\n");
    }

    return 0;
}
```

Output

```
ADD 10 20
30
```

Additional Test Cases

The program was also tested with the other menu options and with an invalid option.

Input	Output	Remark
`ADD 10 20`	`30`	10 + 20
`SUBTRACT 50 15`	`35`	50 - 15
`MULTIPLY 6 7`	`42`	6 x 7
`DIVIDE 4 2`	`Invalid operation`	Not in the menu

4. Conclusion

Through these programs, the concept of enumeration in C was studied. The first program showed how explicit values can be assigned to enum constants, and the second program showed how an enum can represent menu choices and be used with a switch statement to build a simple calculator. Using enumerations makes the code more readable, organised and less error-prone than using plain numeric constants.